

Java Assignment: Functional Interface - Real-World Use Cases

Objective

Design and implement a Functional Interface for a real-world problem. The interface must contain exactly one abstract method and must be implemented using a separate implementation class. Do NOT use Lambda Expressions or anonymous classes.

Instructions

1. Create a Functional Interface with exactly one abstract method.
2. Implement it using a separate implementation class.
3. Do not use Lambda Expressions.
4. Build a console-based application.
5. Follow proper naming conventions and OOP principles.
6. Demonstrate your program using suitable test cases.

Use Case 1: OTP Verification System

Develop a console application that verifies whether the OTP entered by the user matches the generated OTP. Create a Functional Interface named OTPValidator and design the abstract method yourself.

Use Case 2: Payment Verification System

Develop a payment verification application. Create a Functional Interface named PaymentValidator and design an appropriate abstract method to validate a payment before processing.

Use Case 3: Age Verification System

Develop an application that checks whether a user is eligible for registration based on age. Create a Functional Interface named AgeValidator and design the abstract method yourself.

Expected Project Structure

Functional Interface → Implementation Class → Main Class

Example:

OTPValidator.java

OTPValidatorImpl.java

OTPVerificationSystem.java

Constraints

- Exactly one abstract method.
- Students must design the abstract method themselves.
- Do not use Lambda Expressions.
- Do not use anonymous inner classes.
- Implement using a separate class.
- Console-based application only.

Evaluation Criteria

Criteria	Marks
Functional Interface Design	2
Implementation Class	2
Business Logic	2
User Input & Output	2
Code Quality & Naming	2
Total	10

Learning Outcomes

- Understand Functional Interfaces.
- Design meaningful abstract methods.
- Implement interfaces using concrete classes.
- Apply interfaces to real-world scenarios.
- Build simple console-based Java applications.

Note

Students should define their own abstract method (method name, parameters, and return type) based on the selected use case instead of copying a predefined method signature.